```
# Install necessary libraries
!pip install chromadb
!pip install sentence-transformers
```

```
Collecting chromadb
  Downloading chromadb-1.5.6-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (
Collecting build>=1.0.3 (from chromadb)
  Downloading build-1.4.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: pydantic>=2.0 in /usr/local/lib/python3.12/dist-packages (from ch
Requirement already satisfied: pydantic-settings>=2.0 in /usr/local/lib/python3.12/dist-packages
Collecting pybase64>=1.4.1 (from chromadb)
  Downloading pybase64-1.4.3-cp312-cp312-manylinux1_x86_64.manylinux2014_x86_64.manylinux_2_17_x
Requirement already satisfied: uvicorn>=0.18.3 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: numpy>=1.22.5 in /usr/local/lib/python3.12/dist-packages (from ch
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.12/dist-packag
Collecting onnxruntime>=1.14.1 (from chromadb)
  Downloading onnxruntime-1.24.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.met
Requirement already satisfied: opentelemetry-api>=1.2.0 in /usr/local/lib/python3.12/dist-packag
Collecting opentelemetry-exporter-otlp-proto-grpc>=1.2.0 (from chromadb)
  Downloading opentelemetry_exporter_otlp_proto_grpc-1.40.0-py3-none-any.whl.metadata (2.6 kB)
Requirement already satisfied: opentelemetry-sdk>=1.2.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: tokenizers>=0.13.2 in /usr/local/lib/python3.12/dist-packages (fr
Collecting pypika>=0.48.9 (from chromadb)
  Downloading pypika-0.51.1-py2.py3-none-any.whl.metadata (51 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 52.0/52.0 kB 2.6 MB/s eta 0:00:00
Requirement already satisfied: tqdm>=4.65.0 in /usr/local/lib/python3.12/dist-packages (from chr
Requirement already satisfied: overrides>=7.3.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: importlib-resources in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: grpcio>=1.58.0 in /usr/local/lib/python3.12/dist-packages (from c
Collecting bcrypt>=4.0.1 (from chromadb)
  Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl.metadata (10 kB)
Requirement already satisfied: typer>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from chr
Collecting kubernetes>=28.1.0 (from chromadb)
  Downloading kubernetes-35.0.0-py2.py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: tenacity>=8.2.3 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: pyyaml>=6.0.0 in /usr/local/lib/python3.12/dist-packages (from ch
Requirement already satisfied: mmh3>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from chro
Requirement already satisfied: orjson>=3.9.12 in /usr/local/lib/python3.12/dist-packages (from c
Requirement already satisfied: httpx>=0.27.0 in /usr/local/lib/python3.12/dist-packages (from ch
Requirement already satisfied: rich>=10.11.0 in /usr/local/lib/python3.12/dist-packages (from ch
Requirement already satisfied: jsonschema>=4.19.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: packaging>=24.0 in /usr/local/lib/python3.12/dist-packages (from
Collecting pyproject_hooks (from build>=1.0.3->chromadb)
  Downloading pyproject_hooks-1.2.0-py3-none-any.whl.metadata (1.3 kB)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx>=0.2
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx>=0
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from ht
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx>=0.27
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpco
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from js
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: six>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from kuber
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: websocket-client!=0.40.0,!=0.41.*,!=0.42.*,>=0.32.0 in /usr/local
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kuberne
Requirement already satisfied: requests-oauthlib in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: urllib3!=2.6.0,>=1.24.2 in /usr/local/lib/python3.12/dist-package
Collecting durationpy>=0.7 (from kubernetes>=28.1.0->chromadb)
  Downloading durationpy-0.10-py3-none-any.whl.metadata (340 bytes)
```

```
# Import libraries
import chromadb
from sentence_transformers import SentenceTransformer
```

```
client = chromadb.Client()
collection = client.create_collection(name="rag_collection", get_or_create=True)
print(f"ChromaDB collection '{collection.name}' initialized.")
```

```
ChromaDB collection 'rag_collection' initialized.
```

```
model = SentenceTransformer('all-MiniLM-L6-v2')
print("SentenceTransformer model 'all-MiniLM-L6-v2' loaded.")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingfa
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or data
  warnings.warn(
modules.json: 100%  349/349 [00:00<00:00, 36.6kB/s]
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 12.1kB/s]
README.md: 10.5k/? [00:00<00:00, 889kB/s]
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 870B/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF H
config.json: 100%  612/612 [00:00<00:00, 15.1kB/s]
model.safetensors: 100%  90.9M/90.9M [00:01<00:00, 209MB/s]
Loading weights: 100%  103/103 [00:00<00:00, 379.47it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expe
tokenizer_config.json: 100%  350/350 [00:00<00:00, 10.9kB/s]
vocab.txt: 232k/? [00:00<00:00, 5.16MB/s]
tokenizer.json: 466k/? [00:00<00:00, 15.9MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 1.56kB/s]
config.json: 100%  190/190 [00:00<00:00, 4.59kB/s]
SentenceTransformer model 'all-MiniLM-L6-v2' loaded.
```

```
documents = [
    "Artificial Intelligence is the simulation of human intelligence.",
    "Machine Learning is a subset of AI.",
    "Deep Learning uses neural networks.",
    "Natural Language Processing deals with text data.",
    "RAG combines retrieval and generation."
]

ids = ["doc1", "doc2", "doc3", "doc4", "doc5"]
```

```
print("Sample documents prepared.")
```

```
Sample documents prepared.
```

```
embeddings = model.encode(documents)
print(f"Converted {len(documents)} documents into {embeddings.shape[1]}-dimensional embeddings."
```

```
Converted 5 documents into 384-dimensional embeddings.
```

```
collection.add(
    documents=documents,
    embeddings=embeddings.tolist(), # Convert numpy array to list for ChromaDB
    ids=ids
)
print(f"Added {len(documents)} documents and their embeddings to ChromaDB collection '{collectio
```

```
Added 5 documents and their embeddings to ChromaDB collection 'rag_collection'.
```

```
query = "What is AI?"

# Encode the query into an embedding
query_embedding = model.encode([query])

# Perform similarity search in ChromaDB
results = collection.query(
    query_embeddings=query_embedding.tolist(), # Convert numpy array to list
    n_results=2 # Retrieve the top 2 most similar documents
)

print("Similarity search results:")
print(results)
```

```
Similarity search results:
{'ids': [['doc1', 'doc2']], 'embeddings': None, 'documents': [['Artificial Intelligence is the si
```

```
print("Retrieved Documents:")
for doc in results['documents'][0]:
    print(f"- {doc}")
```

```
Retrieved Documents:
- Artificial Intelligence is the simulation of human intelligence.
- Machine Learning is a subset of AI.
```

```
def retrieve(query):
    query_embedding = model.encode([query])
    results = collection.query(
        query_embeddings=query_embedding.tolist(),
        n_results=2 # You can adjust this number
    )
    return results['documents'][0]

print("Testing retrieval function for 'Explain machine learning':")
retrieved_ml_docs = retrieve("Explain machine learning")
for doc in retrieved_ml_docs:
    print(f"- {doc}")
```

```
Testing retrieval function for 'Explain machine learning':
- Machine Learning is a subset of AI.
- Deep Learning uses neural networks.
```